

WorkShield

시스템 아키텍처 구성 보고서

프로젝트: SK Networks Family AI Camp 30기 4차 프로젝트

팀: 2팀

작성 기준일: 2026-07-31

목차

1. 시스템 아키텍처 개요
2. 전체 시스템 아키텍처
3. 애플리케이션 배치 아키텍처
4. 네트워크 및 보안 아키텍처
5. 데이터 및 스토리지 아키텍처
6. LLM 및 MCP 연동 아키텍처
7. 배포 및 운영 아키텍처
8. 주요 설계 판단과 제약사항
9. 결론

부록. 저장소 근거 자료

1. 시스템 아키텍처 개요

1.1 문서 목적과 범위

본 문서는 WorkShield 웹 애플리케이션을 구성하는 기술 요소와 배치 관계를 설명한다. 요구사항 정의서, 화면설계서, 테스트 계획 및 결과 보고서와 중복되지 않도록 AWS 인프라, EC2 컨테이너, RunPod 추론 자원, 저장소, 네트워크 보안, CI/CD 및 운영 책임 경계를 중심으로 기술한다.

1.2 아키텍처 설계 목표

- CloudFront를 단일 진입점으로 사용하여 정적 Web과 API 요청을 경로별로 분리한다.
- 정적 Web은 Private S3에 배치하고 API와 MCP는 단일 EC2의 Docker Compose로 실행한다.
- GPU가 필요한 LLM 및 Embed/Rerank 추론은 RunPod로 분리한다.
- 사용자 데이터와 MCP 검색 코퍼스를 서로 다른 암호화 EBS 볼륨으로 분리한다.
- GitHub Actions의 애플리케이션 배포와 로컬 운영자의 인프라·RunPod 제어를 분리한다.
- OIDC, SSM, Secrets Manager와 CloudFront Origin 제한으로 장기 키와 직접 서버 접근을 최소화한다.

1.3 기술 스택

구분	기술/서비스	주요 역할
Frontend	React, Vite	브라우저에서 실행되는 정적 웹 애플리케이션
Edge	AWS CloudFront	HTTPS, 경로 라우팅, 정적 파일 캐싱
Static Hosting	Amazon S3	Private Web Bucket, Versioning, Release/Live 분리
Compute	Amazon EC2 / Amazon Linux 2023	Nginx, API, MCP 컨테이너 실행
Container	Docker, Docker Compose	단일 호스트 내 서비스 배치
Reverse Proxy	Nginx	Origin TLS, Header 검증, SSE 전달
Application	FastAPI, FastMCP, LangGraph	세션, 작업 상태, MCP/LLM 오케스트레이션
Review Engine	MCP Server	표준조항 검색, 재정렬, 법령 근거 조회
LLM, Retrieval AI Serving	RunPod, vLLM, Embed/Rerank	LLM 추론 Endpoint, 임베딩 생성 및 후보 재정렬
Storage	EBS, SQLite	임시 사용자 데이터, 작업 원장, 검색 코퍼스
Deployment	GitHub Actions, GHCR, AWS SSM	Digest 기반 컨테이너 배포
IaC	AWS CDK	Access, Foundation, Service Stack 프로비저닝

2. 전체 시스템 아키텍처

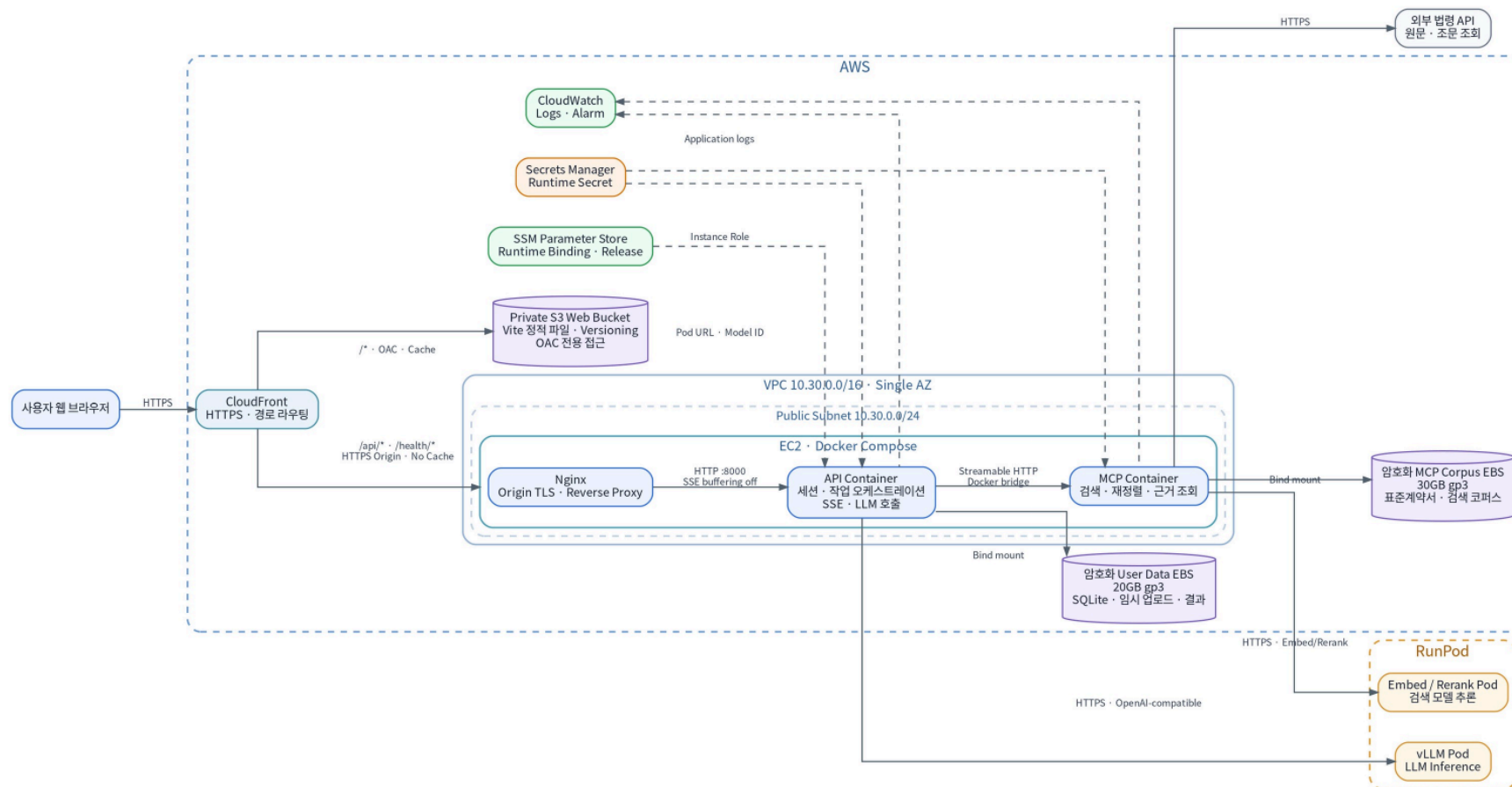


그림 1. WorkShield 전체 시스템 아키텍처

2.1 구성 영역과 책임

영역	구성 요소	책임
Client	사용자 웹 브라우저	CloudFront 도메인에 HTTPS 요청을 전송하고 Vite Web을 실행
AWS Edge	CloudFront	정적 Web과 API Origin을 분리하고 Viewer HTTPS를 강제
Static Web	Private S3 Web Bucket	/live Prefix 정적 파일 제공, OAC 요청만 허용
Application Host	EC2 + Docker Compose	Nginx, API, MCP의 단일 호스트 Runtime
Application	API Container	세션·작업 상태 관리, MCP/LLM 호출 조정
Review	MCP Container	검색 코퍼스와 외부 추론·법령 API 조합
AI Plane	RunPod vLLM / Embed-Rerank	GPU 기반 생성 및 검색 추론
Data Plane	User Data EBS / MCP Corpus EBS	사용자 데이터와 기준 코퍼스 물리 분리
Control Plane	SSM / Secrets Manager / CloudWatch	설정, 비밀, 배포 실행, 로그 및 Alarm

2.2 핵심 기술 흐름

- CloudFront 기본 Cache Behavior는 Private S3 Web Origin을 사용한다.
- /api/*와 /health/*는 캐시를 비활성화하고 EC2 Nginx Origin으로 전달한다.
- Nginx는 CloudFront Origin Header를 검증한 뒤 API Container의 8000 포트로 전달한다.
- API는 Docker Bridge Network에서 MCP Streamable HTTP Endpoint를 호출한다.
- API는 RunPod vLLM, MCP는 Embed/Rerank와 외부 법령 API를 호출한다.
- Runtime Secret은 Secrets Manager, 비밀이 아닌 Binding과 Release 정보는 SSM Parameter Store에 둔다.

3. 애플리케이션 배치 아키텍처

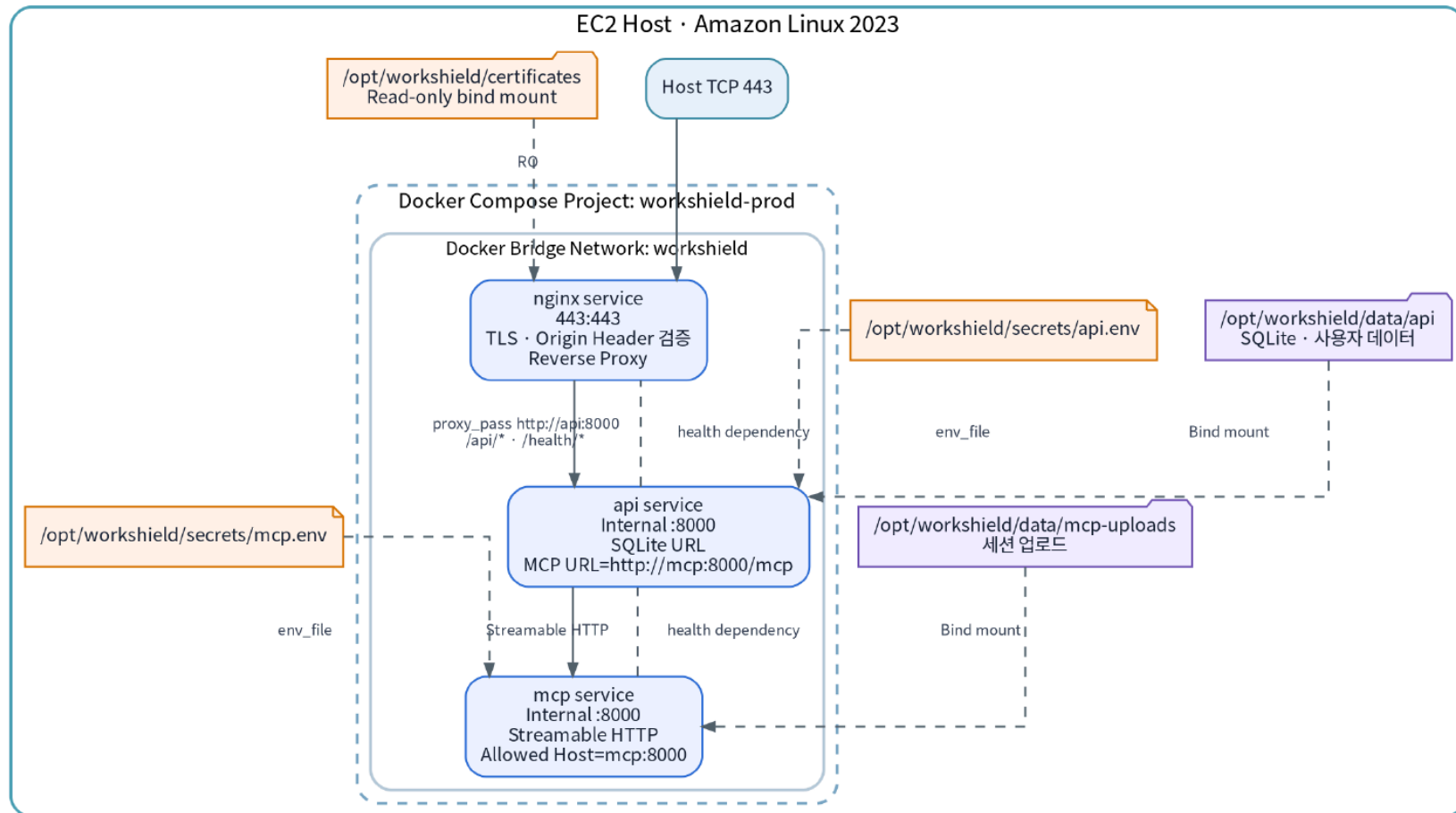


그림 2. EC2 Docker Compose 내부 배치 구조

3.1 Docker Compose 서비스

서비스	네트워크/포트	주요 설정	Health 및 의존성
nginx	Host 443 -> Container 443	Origin TLS, X-WorkShield-Origin 검증, Reverse Proxy	api healthy 이후 시작
api	Docker 내부 8000	APP_ENV=prod, SQLite URL, MCP URL	/health/ready, mcp healthy 이후 시작
mcp	Docker 내부 8000	streamable-http, allowed host mcp:8000	TCP 8000 연결 검사

3.2 Nginx Origin 계약

항목	설정	설계 의도
TLS	TLS 1.2 / 1.3	CloudFront-Origin 전송 암호화
업로드 크기	client_max_body_size 12m	API 10MB와 Multipart Overhead 반영
Body Timeout	120s	파일 업로드 시간 확보
API Timeout	read/send 3600s	장시간 검토와 SSE 유지
Buffering	off	SSE Event 지연 방지
Origin 검증	Header 불일치 시 403	CloudFront 우회 방어
Fallback	location / -> 404	Origin의 임의 경로 차단

4. 네트워크 및 보안 아키텍처

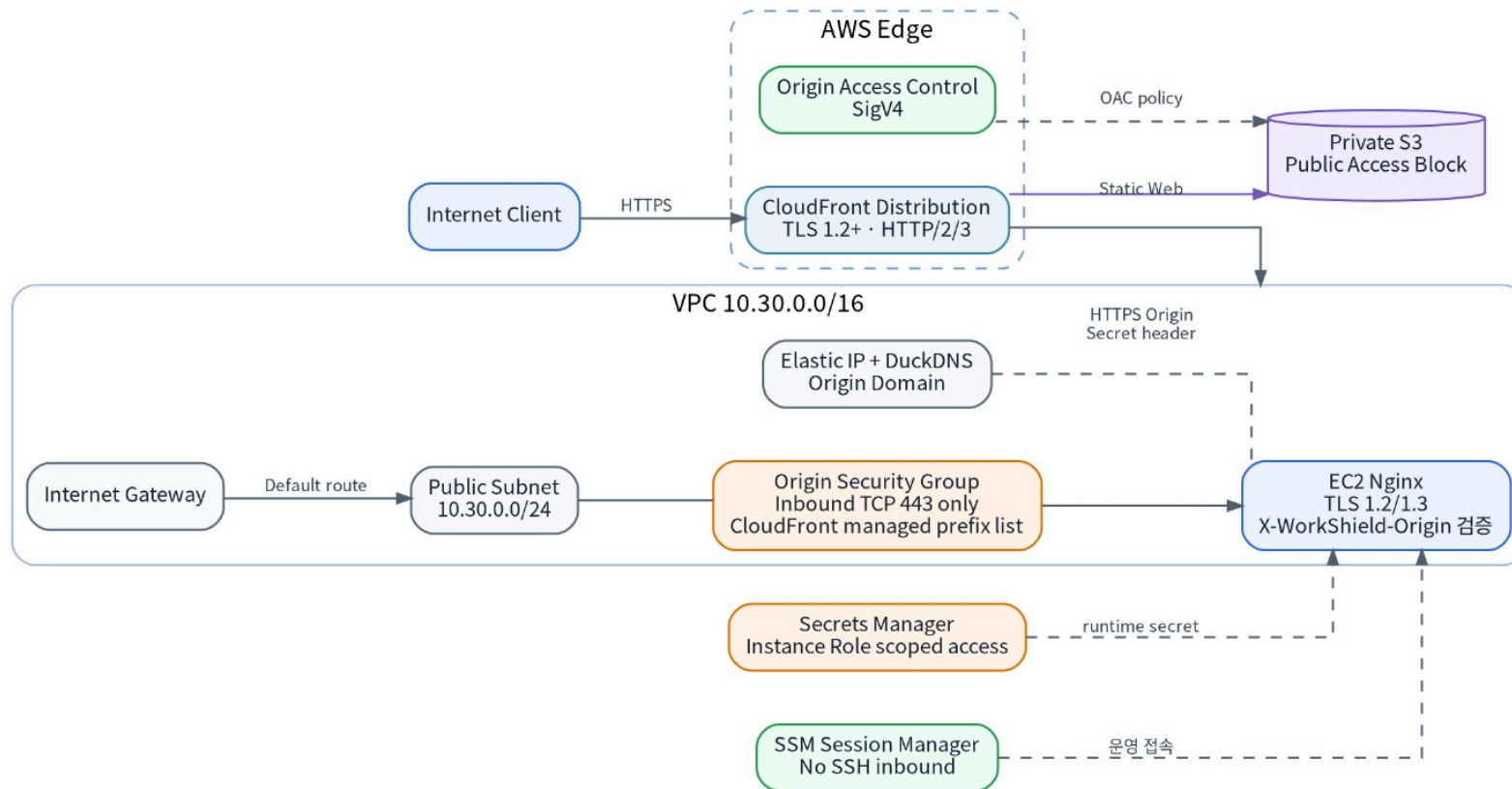


그림 3. 네트워크 경계 및 Origin 보호 구조

4.1 네트워크 자원

자원	현재 구성	비고
VPC	10.30.0.0/16	DNS Hostname 및 DNS Support 활성화
Subnet	Public Subnet 10.30.0.0/24	단일 Availability Zone
Routing	Internet Gateway + 0.0.0.0/0	외부 RunPod/API 통신
Origin Address	Elastic IP + DuckDNS	CloudFront Custom Origin
Security Group	CloudFront Prefix List TCP 443	일반 인터넷 직접 접근 미허용
Management	SSM Session Manager	SSH Inbound 불필요

4.2 CloudFront 라우팅

Path Pattern	Target Origin	Cache	주요 정책
Default /*	Private S3 /live	Caching Optimized	HTTPS Redirect, OAC SigV4, SPA Rewrite
/api/*	EC2 Nginx	Disabled	All Viewer Except Host Header
/health/*	EC2 Nginx	Disabled	API Live/Ready 상태 조회

4.3 보안 경계

- 사용자-CloudFront-EC2 Origin 전 구간 HTTPS 사용
- S3 Public Access 차단 및 Distribution SourceArn 조건 적용
- EC2 IMDSv2 필수, Instance Role 접근 경로 제한
- CloudFront Prefix List와 Nginx Origin Header를 결합
- 운영 접속은 SSM Session Manager, 배포는 고정 SSM Document 사용

5. 데이터 및 스토리지 아키텍처

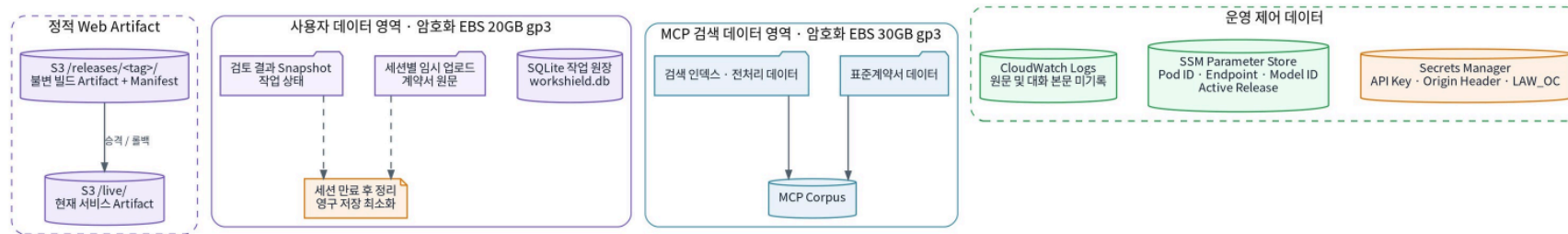


그림 4. 데이터 저장소와 수명주기 분리

5.1 저장소 구성

저장소	데이터	정책
S3 Web Bucket	Vite 정적 파일, Release Manifest	Private, SSL 강제, Versioning, /releases와 /live 분리
User Data EBS 20GB gp3	SQLite, 세션 업로드, 결과 Snapshot	암호화, /opt/workshield/data Mount
MCP Corpus EBS 30GB gp3	표준계약서, 검색 코퍼스, 인덱스	암호화, /opt/workshield/mcp-corpus Mount
SQLite	검토 작업 원장과 결과 상태	단일 EC2 MVP 로컬 Transaction 저장소
Secrets Manager	호출키, Origin Header, 법령 API Key	Instance Role로 Runtime 조회
SSM Parameter Store	Pod ID, URL, Model ID, Release	비밀이 아닌 운영 Binding
CloudWatch Logs	API/MCP Application Log	원문 계약서와 대화 본문 미기록

5.2 운영 파라미터

Parameter	현재 값	용도
/workshield/prod/api/session-ttl-seconds	1800	세션 TTL 30분
/workshield/prod/api/max-upload-size-bytes	10485760	API 업로드 최대 10MB
/workshield/prod/release/active-tag	Runtime 갱신	활성 Release 식별
/workshield/prod/release/active-api-image	Digest	활성 API Image
/workshield/prod/release/active-mcp-image	Digest	활성 MCP Image
/workshield/prod/runtime/binding	JSON	RunPod Endpoint, Pod ID, Model ID

6. LLM 및 MCP 연동 아키텍처

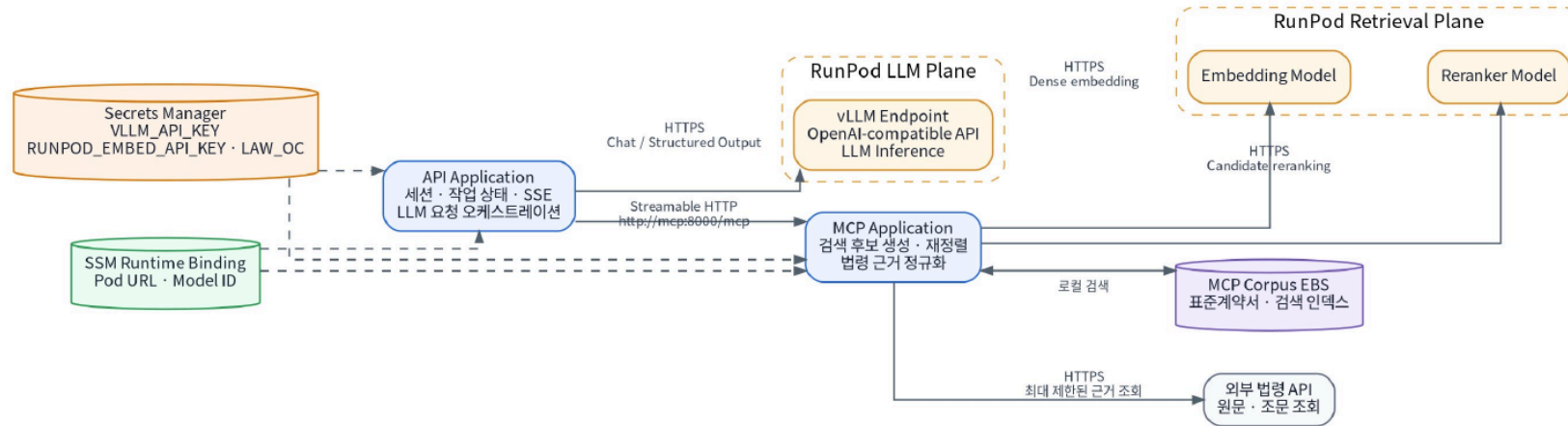


그림 5. API, MCP, RunPod 및 외부 법령 API 연동 구조

6.1 연동 책임 경계

구성 요소	수행 책임	수행하지 않는 책임
API	세션, 작업 상태, SSE, MCP/LLM 호출, 구조화 출력 검증	표준조항 검색 모델 직접 실행
MCP	검색 후보 생성, Embed/Rerank 호출, Corpus 접근, 법령 근거 정규화	사용자 세션과 화면 상태 관리
vLLM Pod	RAG 근거 답변 생성	검토 작업 상태 관리
Embed/Rerank Pod	Embedding 및 후보 재정렬	최종 사용자 답변 생성
외부 법령 API	법령 원문 및 조문 조회	계약서 검토 상태 관리

6.2 통신 및 인증

호출 주체	대상	통신	인증/설정
API	MCP Container	Docker Bridge Streamable HTTP	MCP_ALLOWED_HOSTS=mcp:8000
API	RunPod vLLM	HTTPS, OpenAI-compatible	VLLM_API_KEY + Runtime Binding
MCP	RunPod Embed/Rerank	HTTPS	RUNPOD_EMBED_API_KEY + Runtime Binding
MCP	외부 법령 API	HTTPS	LAW_OC Secret

RunPod 관리 키는 Pod Lifecycle 권한을 가지므로 EC2 Runtime과 GitHub Actions에 전달하지 않는다. Runtime Container에는 Endpoint 호출에 필요한 최소 인증키만 주입한다.

7. 배포 및 운영 아키텍처

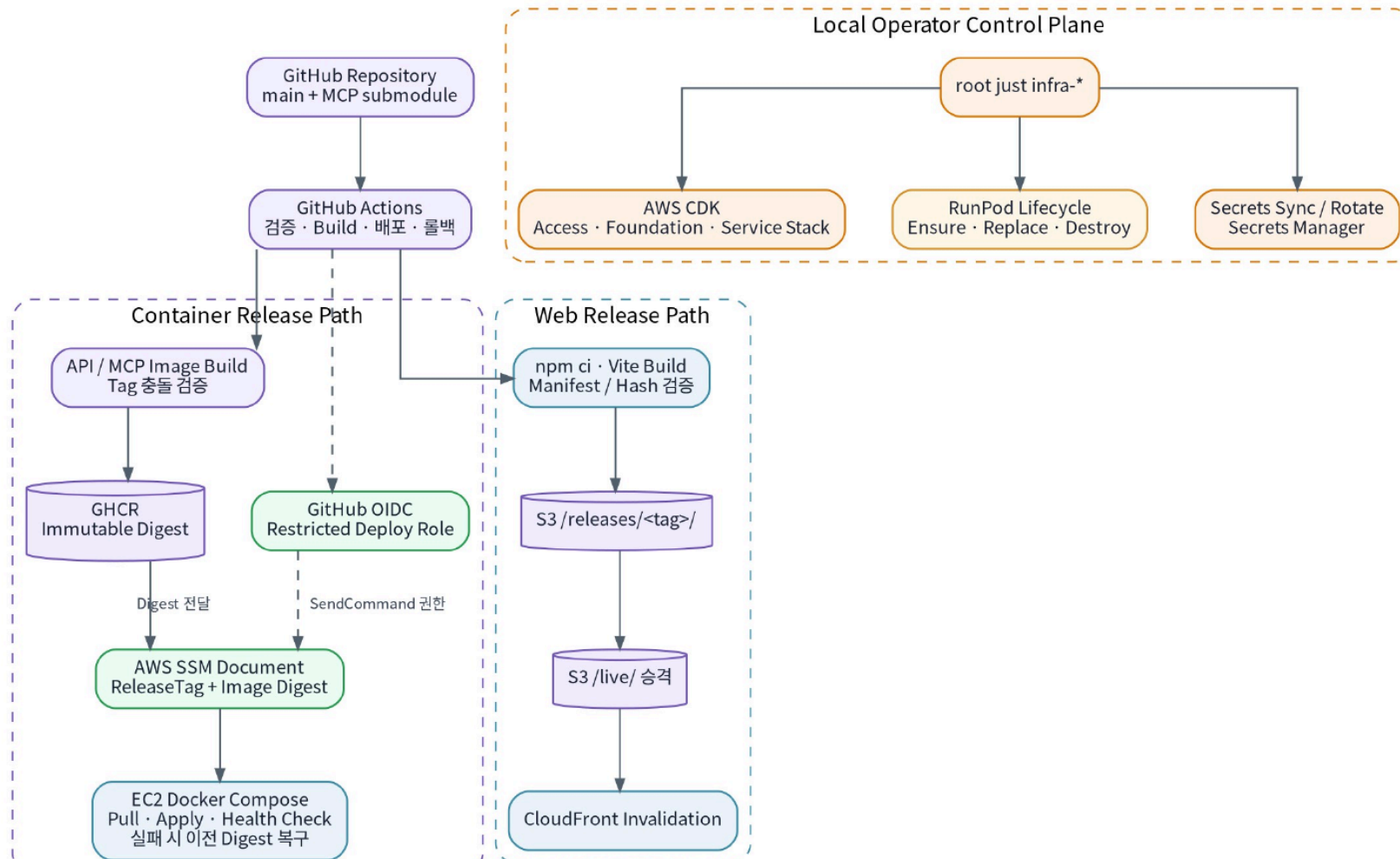


그림 6. Container, Web 및 Infrastructure Control Plane 분리

7.1 Container Release Path

- GitHub Actions가 API와 MCP Image를 Build하고 논리 Tag 충돌을 검증한다.
- Image를 GHCR에 게시하고 운영 배포 시 Tag를 SHA-256 Digest로 해석한다.
- GitHub OIDC로 제한된 Deploy Role을 Assume한다.
- 고정 SSM Document에 Release Tag와 API/MCP Digest만 전달한다.
- EC2가 Image를 Pull하고 Docker Compose를 갱신한다.
- Health 확인 후 활성 Digest를 기록하며 실패 시 이전 Digest를 재적용한다.

7.2 Web Release Path

- GitHub Actions가 npm ci와 Vite Build를 수행한다.
- Artifact와 Manifest를 S3 /releases/<tag>/에 게시한다.
- 동일 Tag의 Commit 또는 Content Hash가 다르면 실패한다.
- 검증된 Release를 /live로 승격하고 CloudFront Invalidation을 수행한다.
- 롤백은 Rebuild 없이 기존 Release Artifact를 다시 /live로 승격한다.

7.3 책임 분리

주체	허용 책임	제한
로컬 운영자	AWS bootstrap, CDK deploy/destroy, RunPod Lifecycle, Secret Sync/Rotate	GitHub Workflow가 대신 수행하지 않음
GitHub CI	Test, Lint, Build 검증, Compose Config, CDK Synth	AWS/RunPod 생성·삭제 없음
GitHub Container Deploy	Digest 확인, OIDC, SSM, Health 및 롤백	CloudFormation·EC2·IAM 변경 및 Secret 조회 불가
GitHub Web Deploy	S3 Release 승격·롤백, CloudFront Invalidation	Container와 RunPod 변경 없음
EC2 Instance Role	지정 Secrets/SSM 조회, CloudWatch 전송	광범위한 계정 관리 권한 없음

8. 주요 설계 판단과 제약사항

8.1 주요 설계 판단

설계 결정	기술적 근거	효과
CloudFront 단일 진입점	S3와 API Origin을 경로로 분리	Client 구성 단순화와 Origin 노출 제한
Private S3 정적 호스팅	Web 서버 Container 불필요	운영 부담과 비용 감소
단일 EC2 Docker Compose	MVP 규모와 기간에 적합	초기 배포와 장애 분석 단순화
API/MCP 논리 분리	사용자 상태와 검색·근거 책임 구분	코드 및 Container 경계 명확화
RunPod GPU 분리	EC2 GPU 상시 운영 회피	GPU 비용과 인프라 복잡도 절감
EBS 볼륨 분리	사용자 데이터와 Corpus 수명 차이	데이터 격리
OIDC + SSM 배포	장기 Key와 SSH 제거	Credential 노출면 축소
Immutable Digest	Mutable Tag 위험 제거	재현 가능한 배포와 롤백

8.2 현재 제약사항

제약	영향	향후 개선 방향
Single AZ / Single EC2	EC2 장애 시 API와 MCP 동시 중단	Multi-AZ, ECS 또는 Host 분리
Public Subnet Origin	Origin이 Public Address 보유	ALB, Private Subnet 검토
SQLite	다중 Instance 수평 확장 제한	RDS PostgreSQL 등으로 이전
동기식 외부 추론	RunPod 지연과 장애 영향	Queue/Worker, Circuit Breaker 강화
외부 법령 API 의존	근거 조회 가용성 영향	Cache, Fallback, 장애 모니터링
로컬 운영자 Control Plane	운영자 환경 의존	승인형 관리 Pipeline 검토

8.3 운영 및 모니터링 기준

- EC2 CPU 5분 평균 80% 이상이 3회 연속이면 High CPU Alarm
- API Live/Ready, MCP Health, API-MCP 호출, RunPod Readiness 확인
- SSE Buffering 비활성화와 CloudFront Origin Read Timeout 120초
- CloudWatch에 계약서 원문, 대화 본문, API Key, Origin Header 미기록
- Workflow와 SSM에는 Image Digest와 비밀이 아닌 Release 정보만 전달

9. 결론

WorkShield는 CloudFront를 단일 Edge 진입점으로 사용하고 Private S3의 정적 Web과 EC2의 API Origin을 경로별로 분리한다. EC2 내부에는 Nginx, API, MCP를 Docker Compose로 배치하고 GPU가 필요한 LLM과 Embed/Rerank 추론은 RunPod로 분리한다.

데이터 계층에서는 사용자 데이터와 MCP 검색 코퍼스를 별도 암호화 EBS에 배치하고 Runtime Secret과 운영 Binding을 Secrets Manager와 SSM Parameter Store로 분리한다. 배포 계층에서는 GitHub Actions가 검증된 Artifact만 기존 인프라에 반영하고 AWS CDK 및 RunPod Lifecycle은 로컬 운영자가 관리한다.

부록. 저장소 근거 자료

Ref	저장소 경로	확인 내용
R1	docs/architecture/infra/README.md	전체 운영 인프라 구조와 책임 경계
R2	infra/README.md	Control Plane 디렉터리와 Stack 구분
R3	infra/src/workshield_infra/stacks/foundation.py	VPC, Subnet, SG, IAM, EBS, SSM, CloudWatch
R4	infra/src/workshield_infra/stacks/service.py	EC2, S3, CloudFront, OAC, SSM Document
R5	infra/src/workshield_infra/stacks/access.py	GitHub OIDC Deploy Role 경계
R6	infra/assets/compose.yaml	Nginx/API/MCP Service, Network, Health
R7	infra/assets/nginx.conf.template	Origin TLS, Header, Timeout, SSE
R8	docs/architecture/operations/deploy.md	Container/Web 배포 및 롤백
R9	docs/architecture/operations/secrets.md	Secret 저장, Sync/Rotate, GitHub 제한
R10	docs/architecture/시스템_구성도_초안.svg	기존 구성도 표현 요소